

Hypergraph clustering

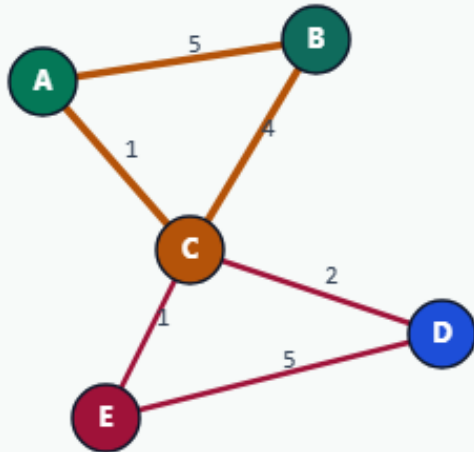
Netlists are hypergraphs, one net can touch many pins

Nets are hyperedges

HYPERGRAPH CLUSTERING

Nets are hyperedges

Step 1 / 5 · nets · module03-03-hypergraph-clustering



Explanation

TINY_HYPERGRAPH has four nets: multi-pin $n1=\{A,B,C\}$ weight 3, pair $n2=\{D,E\}$, bridge $n3=\{C,D\}$, and $n4=\{A,B\}$. Cut counts whole nets, not pairs.

- Hyperedge cut if pins span ≥ 2 clusters
- $n1$ pulls ABC together
- Graph clique expansion would look different

4 hyperedges
5 nodes

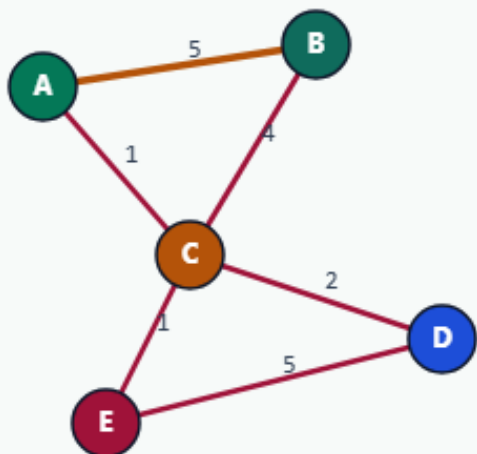
Nets are hyperedges

Affinity = shared pin weight

HYPERGRAPH CLUSTERING

Affinity = shared pin weight

Step 2 / 5 · affinity · module03-03-hypergraph-clustering



Explanation

Greedy merge scores pairs by summed weights of hyperedges that contain both endpoints. A-B and A-C-B affinities favor the ABC community.

- $\text{pair_affinity}(a,b) = \sum w$ over shared nets
- Contract highest affinity pair
- Rewrite pins onto supernodes

priority: co-occurrence on nets

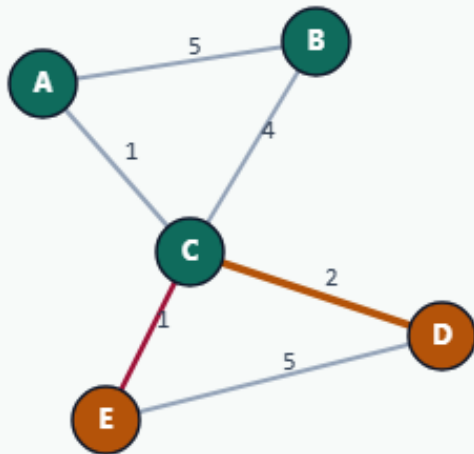
Affinity = shared pin weight

Merge down to K=2

HYPERGRAPH CLUSTERING

Merge down to K=2

Step 3 / 5 · merge-k2 · module03-03-hypergraph-clustering



Explanation

Repeated contraction yields two clusters covering ABC and DE. Only bridge net n3 is cut.

- Target K=2
- ABC stays on one supernode family
- DE on the other

parts: ABC|DE
hyperedge cut: 1

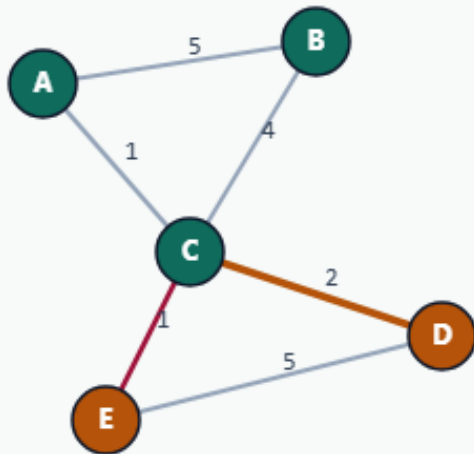
Merge down to K=2

Hyperedge cut = 1

HYPERGRAPH CLUSTERING

Hyperedge cut = 1

Step 4 / 5 · cut-1 · module03-03-hypergraph-clustering



Explanation

Golden: hyperedge cut 1 (n3 only). Pairwise graph cut on a clique expansion would score differently — that is the teaching contrast.

- n1, n2, n4 uncut
- n3 cut → +1
- Objective matches netlist reality

hyperedge cut: 1
parts: ABC|DE

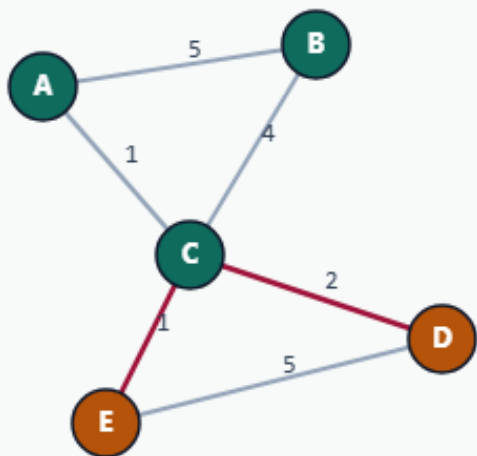
Hyperedge cut = 1

Why hypergraphs in EDA

HYPERGRAPH CLUSTERING

Why hypergraphs in EDA

Step 5 / 5 · takeaway · module03-03-hypergraph-clustering



Explanation

Real nets touch many cells. Modeling them as hyperedges keeps the cut objective honest. Browser view clique-expands only for drawing.

- Cut nets, not just edges
- Multi-pin nets dominate affinity
- Next: congestion / timing objectives

Starter golden: cut=1

Browser lab track

- In the browser lab, load the starter hypergraph and run greedy clustering to K equals two
- Confirm hyperedge cut one for ABC versus DE

Implement track

- Run hypergraph greedy to K equals two
- Confirm hyperedge cut one and the natural clusters

Implement track — try these

```
export PYTHONPATH=./common
```

```
python ../common/solvers.py examples/tiny_hypergraph.json --mode  
hypergraph --k 2
```

Pitfalls to watch

- Clique-expanding hyperedges changes the objective
- Counting a cut once per hyperedge, not per pair, is required
- Empty or singleton nets must be filtered

Your turn

- Match hyperedge cut one